

Apuntes del Curso: Computación Paralela y Distribuida

Prof. José Fiestas

16 de agosto de 2026

En el diseño de la solución computacional paralela a un problema, debemos considerar 3 principales componentes:

- El algoritmo (abstracción y código)
- Alto Performance (medido en métricas como speedup, eficiencia, precisión)
- La topología usada (óptima para el algoritmo)

En el curso, vamos a trabajar en el algoritmo y medición de métricas de performance. La topología se determinará con la herramienta de programación utilizada y no será optimizada.

U1.4 Taxonomía de Flynn

Clasificación de arquitecturas de computadoras propuesta por Michael J. Flynn (1966), basada en el número de flujos de instrucciones y datos que puede procesar una arquitectura en paralelo.

0.1. Single Instruction Single Data

Procesadores secuenciales ejecutan una instrucción sobre una unidad de memoria (Arquitectura Von Neumann es SISD) E.g. microcontroladores

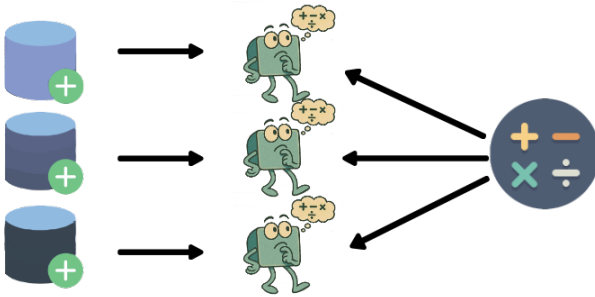
0.2. Single Instruction Multiple Data

Una única instrucción (tarea) se ejecuta en uno o más núcleos en múltiples **data streams**, en forma simultánea (GPUs, arquitecturas vectoriales) Algoritmos: suma de vectores, filtros de imagen

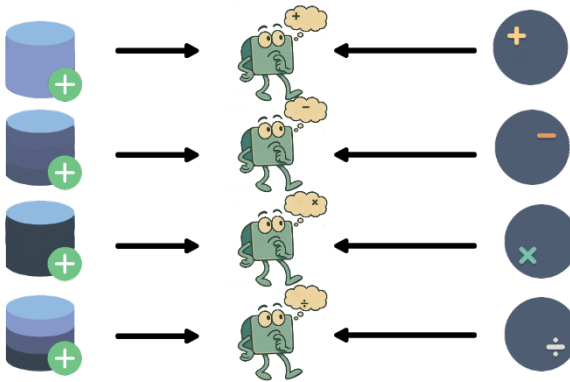
SISD



SIMD



MIMD



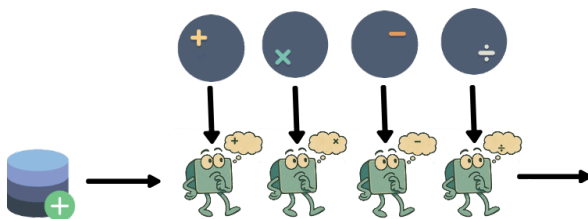
0.3. Multiple Instruction Multiple Data

Múltiples tareas en múltiples procesos se ejecutan en distintos **data streams** en forma simultánea (arquitecturas de memoria compartida o distribuída, GPU modernas en modo GPGPU)
Algoritmos: multiplicación de matrices con hilos independientes, N-Body paralelo,

0.4. Multiple Instruction Single Data

Múltiples procesos ejecutan distintas instrucciones en la misma data. E.g. sistemas tolerantes a fallos en tiempo real (sistemas de navegación aérea),

MISD



U1.5 Diseño en paralelo: método de Foster

El método fue propuesto por Ian Foster, un científico computacional británico-estadounidense, en el contexto de su trabajo en computación paralela durante las décadas de 1980 y 1990. Foster ha sido una figura importante en el desarrollo de sistemas paralelos y distribuidos, y también es coautor del libro clásico *Designing and Building Parallel Programs* (1995), donde se sistematizó este método

0.5. Tasks y channels

En paralelismo, se utilizan dos abstracciones fundamentales:

- **Task**, programa, memoria local y puertos de entrada/salida.
- **Channel**, mensaje que conecta la salida de un task con la entrada de otro. Mecanismo que define la dependencia entre tasks.

Características:

- Un cálculo en paralelo consiste de dos o más tasks, que se ejecutan en forma concurrente/simultánea)
- Cada task es **secuencial** en memoria local
- Tasks realizan operaciones de *lectura*, *escritura*, *envío* y *recibo* de datos, *creación* de nuevos tasks, y *término* de los mismos.
- **envío** de datos es **asíncrono**, **recibo** de datos es **síncrono** (causa bloqueo)
- entrada/salida de datos hacia/desde tasks se conectan a través de colas de mensajes (channels) que pueden ser creadas y destruidas
- tasks pueden ser asignadas a uno o varios procesos

Propiedades:

- Performance, a través del mapeo de tasks a procesos e implementación de channels (comunicación)
- Independencia, resultados de cálculos en tasks son independientes de el proceso en que se realicen
- Modularidad: el modelo task/channel se puede desarrollar en forma modular. Un task encapsula data y cómputo sobre la data, mientras que los puertos de comunicación constituyen una interfase
- Determinismo: generalmente requerida en la ejecución de algoritmos paralelos

0.6. Método de Foster (PCAM)

0.7. Método de Foster: Particionamiento

1. **Particionamiento**, de datos y tareas en piezas (subdominios). Un buen particionamiento separa los datos y tareas en muchas piezas pequeñas. Es decir, se puede aplicar:

- **Descomposición del dominio**, que divide la **data** en piezas y luego determina que tareas se ejecuta en cada pieza. Típicamente se trata de la data de mayor tamaño o a la que se accede con mayor frecuencia
- **Descomposición funcional**, que divide la **tarea** en piezas y luego se determina que data ejecuta cada pieza

En ambos casos, se trata de encontrar la mayor cantidad de piezas (*primitive tasks*) posible, que brindan un límite superior al paralelismo.

- la cantidad de *primitive tasks* es por lo menos un orden de magnitud mayor que la cantidad de recursos (procesos) utilizados (punto de partida para un diseño más sofisticado)
- *primitive tasks* son aproximadamente del mismo tamaño (particionamiento óptimo)
- la cantidad de tasks crece con el tamaño del problema (escalabilidad)

0.8. Método de Foster: Comunicación

2. **Comunicación**, entre tasks definidos en el paso 1. Existen dos tipos de comunicación:

- **Local**, cuando un task necesita información de un número pequeño de otros tasks para ejecutar una tarea
- **Global**, cuando un número significativo de tasks debe interactuar para completar una tarea

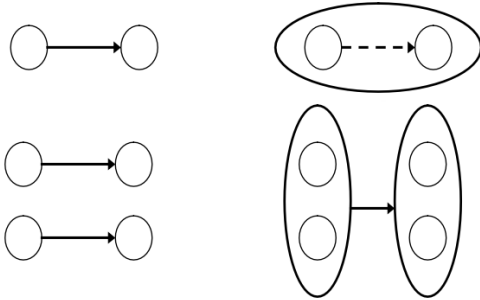
La comunicación se realiza a través de canales abiertos entre tasks. Se considera este procedimiento un *overhead* del algoritmo paralelo ya que no se necesita en un algoritmo secuencial. Por ello, se busca que:

- la comunicación sea balanceada entre tasks
- cada task comunique solo a un número pequeño de vecinos
- tasks puedan comunicar en forma concurrente (simultánea)
- tasks puedan ejecutar tareas en forma concurrente (simultánea)

0.9. Método de Foster: Agloremación

3. Aglomeración, consiste en agrupar tasks con el fin de elevar performance o simplificar la programación. Aglomeración busca:

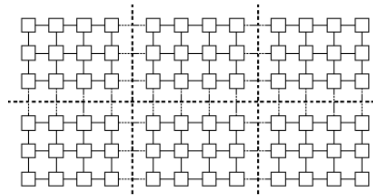
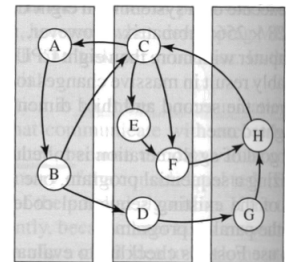
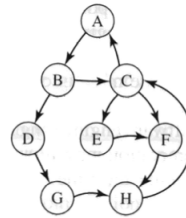
- reducir el *overhead* (i.e. agrupar tasks que implican comunicación, agrupar tasks que no pueden ser concurrentes, agrupar tasks que envían y que reciben para reducir la latencia)
- que el diseño garantice escalabilidad, para aprovechar el máximo de recursos (procesos)



0.10. Método de Foster: Mapping

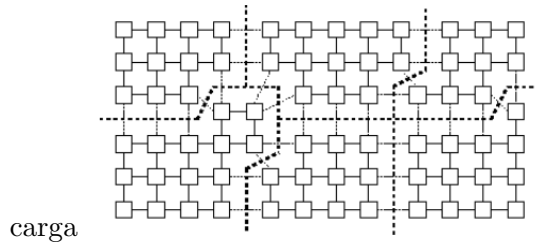
4. Mapping, consiste en asignar tasks a procesos.

- Se busca maximizar la utilización de procesos (cómputo correctamente balanceado) y minimizar la comunicación entre ellos (tasks dependientes se asignan al mismo proceso)
- se considera asignar tanto un task como multitasks por proceso
- se considera la asignación estática y dinámica de tasks a procesos
- balanceo estático: cada task ejecuta el mismo cálculo y se comunica con 4 vecinos. Se distribuye la data



entre 6 procesos equitativamente

- balanceo dinámico: cada proceso recibe un número variable de tasks para compensar desbalance de



1. Estrategia de diseño de un algoritmo en paralelo

El problema de diseño de un algoritmo paralelo se refiere a

- Dado un algoritmo, buscar la arquitectura adecuada en paralelo
- Dada una arquitectura, buscar el algoritmo adecuado en paralelo. Es el caso clásico, que requiere :
 - Identificar **tareas** (tasks) con o por **procesos** (threads)
 - Diseñar un plan de ejecución (**scheduling**), de acuerdo a las dependencias y el I/O
 - Identificar los puntos de **comunicación** entre procesos y del I/O

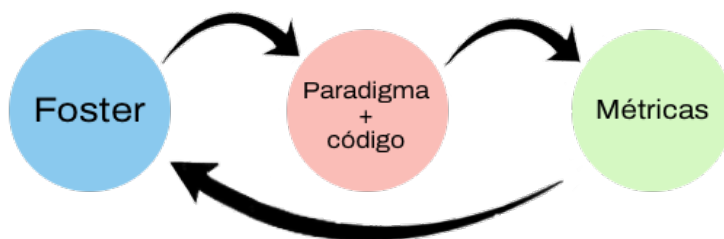
Los principales problemas que aparecen son:

- **Desbalance de cargas**
- **Cuellos de botella**
- **Comunicación**

Método de diseño en paralelo

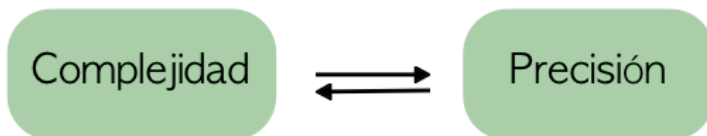
Se proponen los siguientes pasos:

- Aplicar **Foster** (particionar-comunicar-aglomerar-mapear) y escoger el **paradigma** más cercano al problema (compartido, distribuido, híbrido)
- Desarrollar algoritmo de acuerdo al paradigma elegido (pseudocódigo-código)
- Evaluar métricas que definan el grado de **sincronización** y **comunicación** entre procesadores. Un mal diseño de comunicación implica mayor costo computacional, mientras que una mala sincronización origina tiempos muertos (inactividad de procesadores)
- Si el código no es eficiente, retornar al primer paso

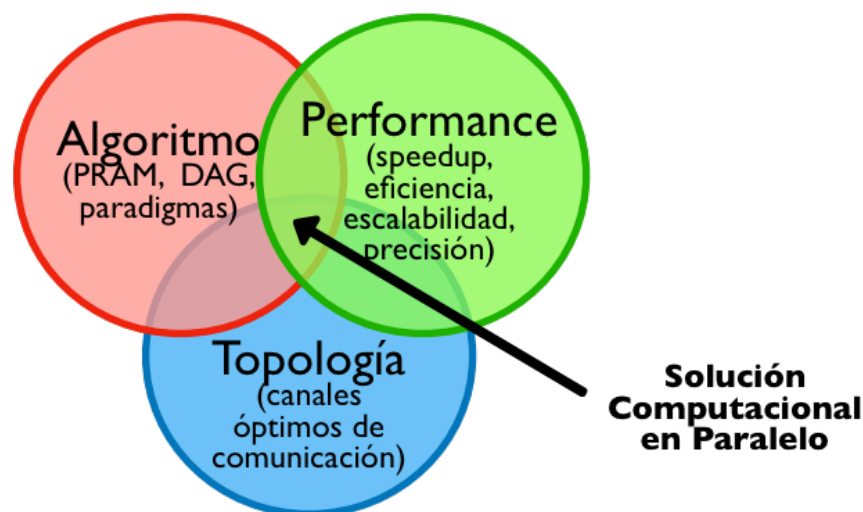


Objetivos (métricas)

- **Primer objetivo:** *minimizar el tiempo total de ejecución*
 - **Tiempo de cálculo** propiamente dicho, se optimiza asignando tareas a procesadores distintos para maximizar la concurrencia
 - **Tiempo de comunicación**, asignando tareas independientes en cada proceso
 - **Tiempo de ocio**, evitar tiempos de inactividad.
- **Segundo objetivo:** mantener (o mejorar) la *precisión del algoritmo* secuencial correspondiente. Este no es un problema trivial



Diseño de una solución computacional en paralelo



Participación 2 (Foster)

Describa los pasos necesarios para paralelizar la suma secuencial de n números, utilizando el Método de Foster. Aplique el método para $n=10^{12}$ e indique la cantidad óptima de procesadores necesarios para paralelizar esta tarea en forma eficiente. Desarrolle los 4 pasos de Foster, y determine:

- Particionamiento: el *primitive task*
- Comunicación: la cantidad de canales de comunicación (inicial)
- Aglomeración: la cantidad final de *channels* (describa el criterio de aglomeración)
- Mapeo: cada proceso puede sumar como máximo 10^9 números

Si la suma de dos números se realiza en un ciclo de 10^{-9} segundos, y el envío/recibo de datos sucede con un ancho de banda de 10^{-8} segundos / Byte, calcule el tiempo total de ejecución $T_{pej} = T_{comp} + T_{comm}$ ¿Es el tiempo en paralelo menor que el secuencial?

Respuesta:

Propuesta 1: suma lineal

- **Particionamiento:** una primera aproximación es la suma de dos números por tarea (task). Asumiendo una topología lineal, tenemos $10^{12} - 1$ tareas. Es decir, la primera tarea suma dos números y las demás realizan la suma acumulada de su número y el resultado de la tarea anterior.
- **Comunicación:** si asumimos una distribución lineal de procesos, se necesitan $10^{12} - 2$ canales de comunicación.
- **Aglomeración:** podemos reducir la cantidad de canales de comunicación, si asignamos más números por tarea. Por ejemplo 10^5 . De esta manera se reducen los canales de comunicación a 10^7 .
- **Mapping:** Ya que cada proceso puede como máximo sumar 10^9 números, se asigna esa cantidad por tarea en cada proceso, y se reduce la complejidad de comunicación a 10^3 canales.

Tiempo Secuencial:

$$T_s = 10^{12} \cdot 10^{-9} = 1000 \text{ sec}$$

(suma acumulada secuencial)

Tiempo Paralelo:

$$T_p = 10^9 \cdot 10^{-9} + 10^3 \cdot 10^{-9} + 10^3 \cdot 8 \cdot 10^{-8} \approx 1,00008 \text{ sec}$$

(10^9 sumas por proceso + 10^3 sumas de vecinos + 10^3 envíos entre procesos)

Propuesta 2: suma por reducción

- **Particionamiento:** una primera aproximación es la suma de dos números por tarea (task). Asumiendo una topología de árbol binario, tenemos $2 \cdot 10^{12} / 2 - 1$ tareas. Es decir, las sumas se realizan por pares de nodos y se va acumulando el resultado en cada nivel.
- **Comunicación:** si asumimos la distribución de árbol binario, se necesitan entonces $10^{12} - 2$ canales de comunicación.
- **Aglomeración:** podemos reducir la cantidad de canales de comunicación, si asignamos más números por tarea. Por ejemplo 10^5 . De esta manera se reducen los canales de comunicación a 10^7 .
- **Mapping:** Ya que cada proceso puede como máximo sumar 10^9 números, se asigna esa cantidad por tarea en cada proceso, y se reduce la complejidad de comunicación a 10^3 canales.

Tiempo Secuencial:

$$T_s = 10^{12} \cdot 10^{-9} = 1000 \text{ sec}$$

(suma acumulada secuencial)

Tiempo Paralelo:

$$T_p = 10^9 \cdot 10^{-9} + 10 \cdot 10^{-9} + 10 \cdot 8 \cdot 10^{-8} \approx 1,0000008 \text{ sec}$$

(10^9 sumas por proceso + 10 sumas de vecinos + 10 envíos entre procesos)

Nota: La cantidad de canales y tareas primitivas, así como el resultado del *mapping* en Foster no se modifica en ambos casos. Sin embargo, la distribución de tareas en paralelo, en el árbol binario, si afecta al calculo del tiempo de ejecución, aunque hay que resaltar que esa diferencia es mínima ya que el término dominante es, en este ejercicio, de cómputo (*compute bound*)

Referencias

- [1] Ian Foster, *Designing and Building Parallel Programs* <https://www.mcs.anl.gov/~itf/dbpp/text/book.html>
- [2] Peter S. Pacheco, *An Introduction to Parallel Programming*. 1st. Morgan Kaufmann, 2011. isbn: 978-0-12-374260- 5.
- [3] Michael J. Quinn, *Parallel Programming in C with MPI and OpenMP*. 1st. McGraw-Hill Education Group, 2003. isbn: 0071232656.